

PLUTUS

75%

PLUTUS

Live

PLUTUS

PROTO

PROTO: SMA Momentum Crossover

SMA(1000) Momentum Crossover on VN30F1M

Live tick-by-tick trading bot for the Vietnamese VN30 Index Futures market

APCS | CS408: Computation Finance Student IDs: 22125009 · 22125038

Abstract

In this project, we implement a **Simple Moving Average (SMA) Momentum Crossover** strategy applied to the **VN30 Index Futures (VN30F1M)** — the front-month futures contract on the Vietnamese stock market. The bot connects to the [AlgoTrade PaperBroker](#) platform via FIX protocol for order execution and Kafka for real-time market data. The portfolio is evaluated on both in-sample and out-of-sample periods to assess its real-world viability.

Introduction

Momentum-based trading strategies seek to exploit the tendency of assets to continue moving in the direction of an established trend. One of the most straightforward momentum indicators is the Simple Moving Average (SMA): when the current price crosses above the SMA, it may signal the beginning of an uptrend; when it crosses below, a downtrend may follow.

This project applies the SMA crossover signal to tick-by-tick data from the VN30 Index Futures market, using disciplined take-profit and stop-loss rules to manage risk. The strategy is fully automated and designed for live paper trading via the [AlgoTrade PaperBroker](#) platform.

Hypothesis

We maintain at most **1 open contract** at any time. Signals are generated by comparing the current tick price to the SMA computed over the last **1000 ticks**:

Signal	Condition
Open Long	$P_{t-1} < SMA_{t-1}$ and $P_t \geq SMA_t$
Open Short	$P_{t-1} > SMA_{t-1}$ and $P_t \leq SMA_t$

Exit rules:

Rule	Condition	Action
Take-Profit	Unrealized P&L $\geq$ +3 points	Close position

Rule	Condition	Action
Stop-Loss	Unrealized P&L $\leq$ -2 points	Close position
End-of-Day	Current time $\geq$ 14:29:55	Force-close all positions (no overnight holds)

Orders are placed as **aggressive limit orders** with a ± 1.0 point slippage to maximise fill probability without crossing circuit limits. Each trade is charged a fee of **0.4 per contract** ($\times 100$ multiplier).

Data

- **Data source:** [AlgoTrade PaperBroker](#)
- **In-sample period:** 2022-01-01 to 2023-01-01
- **Out-of-sample period:** 2024-01-02 to 2025-04-29
- Tick price data for VN30F1M is extracted and stored in **.csv** files to reduce load time for subsequent runs.
- Each buy or sell order incurs a fee of **0.4 per contract**.

Data Collection

Tick price data

- Daily tick-level close prices for VN30F1M are collected from the AlgoTrade database using the script **data_loader.py**.
- Data is stored in the **data/is/** and **data/os/** folders.

Implementation

Environment Setup

1. Set up a Python virtual environment:

```
python -m venv venv
source venv/bin/activate           # Linux / macOS
.\venv\Scripts\activate.bat       # Windows Command Prompt
.\venv\Scripts\Activate.ps1      # Windows PowerShell
```

2. Install the required packages:

```
pip install -r requirements.txt
```

3. Install the PaperBroker client (download the **.whl** from the [PaperBroker docs](#)):

```
pip install paperbroker_client-0.2.4-py3-none-any.whl
```

4. **(Optional)** Create a `.env` file in the project root from the provided template:

```
cp .env.example .env
```

```
PAPER_ACCOUNT_ID=D1
PAPER_USERNAME=<username>
PAPER_PASSWORD=<password>
SENDER_COMP_ID=<sender_comp_id>
TARGET_COMP_ID=SERVER
SOCKET_HOST=papertrade.algotrade.vn
SOCKET_PORT=5001
PAPER_REST_BASE_URL=https://papertrade.algotrade.vn/accounting
PAPERBROKER_KAFKA_BOOTSTRAP_SERVERS=<kafka_bootstrap_servers>
PAPERBROKER_KAFKA_USERNAME=<kafka_username>
PAPERBROKER_KAFKA_PASSWORD=<kafka_password>
PAPERBROKER_ENV_ID=<env_id>
VN30F1M=HNXDS:VN30F2605
```

Data Collection

Option 1 — Download from Google Drive

Download data directly from [Google Drive](#). Place the downloaded folder at the project root:

```
data/
├── is/
└── os/
```

Option 2 — Run the data loader

```
python data_loader.py
```

In-sample Backtesting

Specify the period and parameters in `parameter/backtesting_parameter.json`, then run:

```
python backtesting.py
```

Results are stored in the `result/backtest/` folder.

Optimization

Configure the search space in `parameter/optimization_parameter.json`, then run:

```
python optimization.py
```

The best parameters found are saved to `parameter/optimized_parameter.json`.

Out-of-sample Backtesting

Uses the parameters in `parameter/optimized_parameter.json`. Run:

```
python evaluation.py
```

Results are stored in the `result/optimization/` folder.

Live Paper Trading

```
python paper_trading.py
```

The bot will:

1. Connect and log on to the FIX engine
2. Cancel any orphaned orders from a previous session
3. Subscribe to live market data via Kafka
4. Begin evaluating ticks and trading autonomously
5. Shut down cleanly on `Ctrl+C`

Architecture

```
paper_trading.py
├── LiveSMABot
│   ├── _init_clients()           – Connects FIX (execution) + Kafka (market
data)
│   ├── on_quote_update()        – Tick handler: SMA calc → signal eval →
order routing
│   ├── on_order_filled()        – FIX fill callback: confirms inventory
state
│   ├── on_order_failed()       – FIX reject/cancel callback: frees order
lock
│   ├── open_position()          – Places aggressive limit order to enter
│   ├── close_position()        – Places aggressive limit order to exit
│   └── run()                    – Async main loop: FIX logon → MD
subscribe → event loop
```

Key Design Decisions

- **Pending-order lock** — Only one live order at a time. New signals are suppressed while an order is outstanding.
- **Pending inventory intent** — Inventory state is only updated on confirmed fill (`on_order_filled`), never on order placement.
- **Stuck-order watchdog** — Orders unconfirmed for **120 seconds** are automatically canceled and the lock is freed.
- **Crash recovery** — Orphaned orders from a previous session are loaded from `orders.db` and canceled on startup.
- **Aggressive limit orders** — Entry/exit prices include **±1.0 point slippage** to maximize fill probability without crossing circuit limits.

Project Structure

```

.
├── paper_trading.py           # Live trading bot
├── new_paper_trading.py      # Experimental / WIP version
├── backtesting.py           # Historical backtest engine
├── optimization.py          # Parameter optimization
├── evaluation.py            # Out-of-sample evaluation
├── data_loader.py           # Historical data loader
├── price_util.py            # Price utility helpers
├── utils.py                 # General utilities
├── config/
│   └── config.py            # Configuration constants
├── database/
│   ├── data_service.py
│   └── query.py
├── metrics/
│   └── metric.py            # Sharpe, Sortino, Information ratio, MDD
├── parameter/
│   ├── backtesting_parameter.json
│   ├── optimization_parameter.json
│   └── optimized_parameter.json
├── guides/                  # PaperBroker API usage examples
├── data/                   # Historical data (see Setup)
├── result/
│   ├── backtest/           # In-sample backtest charts
│   └── optimization/       # Out-of-sample backtest charts
├── orders.db               # Auto-generated order store (crash
recovery)
├── .env.example            # Environment variable template
└── CS408_Trading_Algorithm.md # Strategy specification document

```

Strategy Parameters

Parameter	Default	Description
SMA_WINDOW	1000	Rolling tick window for SMA calculation
TP_POINTS	3.0	Take-profit threshold (points)
SL_POINTS	2.0	Stop-loss threshold (points)
END_OF_DAY_CLOSE	14:29:55	Force-close time (5 s before ATC)
FEE_PER_CONTRACT	0.4×100	Round-trip fee per contract

The live trading bot defaults to `SMA_WINDOW = 10` for faster warm-up during live testing sessions.

In-sample Backtesting

Run the in-sample backtesting:

```
python backtesting.py
```

Evaluation Metrics

Backtesting results are stored in `result/backtest/`. The strategy is compared to no-benchmark (futures contract):

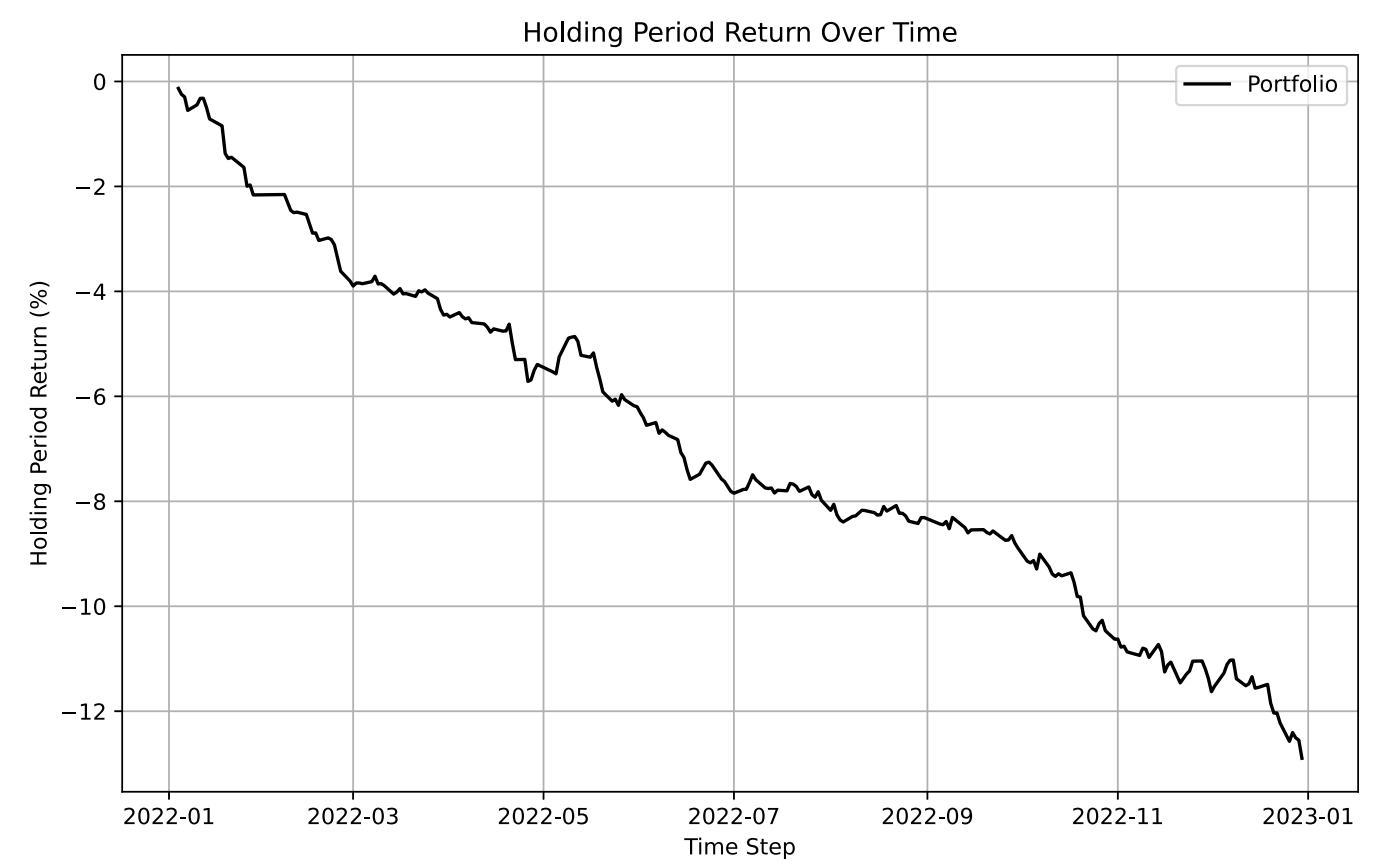
- Sharpe Ratio (SR)
- Sortino Ratio (SoR)
- Information Ratio (IR)
- Maximum Drawdown (MDD)

A risk-free rate of **6% per annum** (~0.023% per day) is used as the benchmark for SR and SoR.

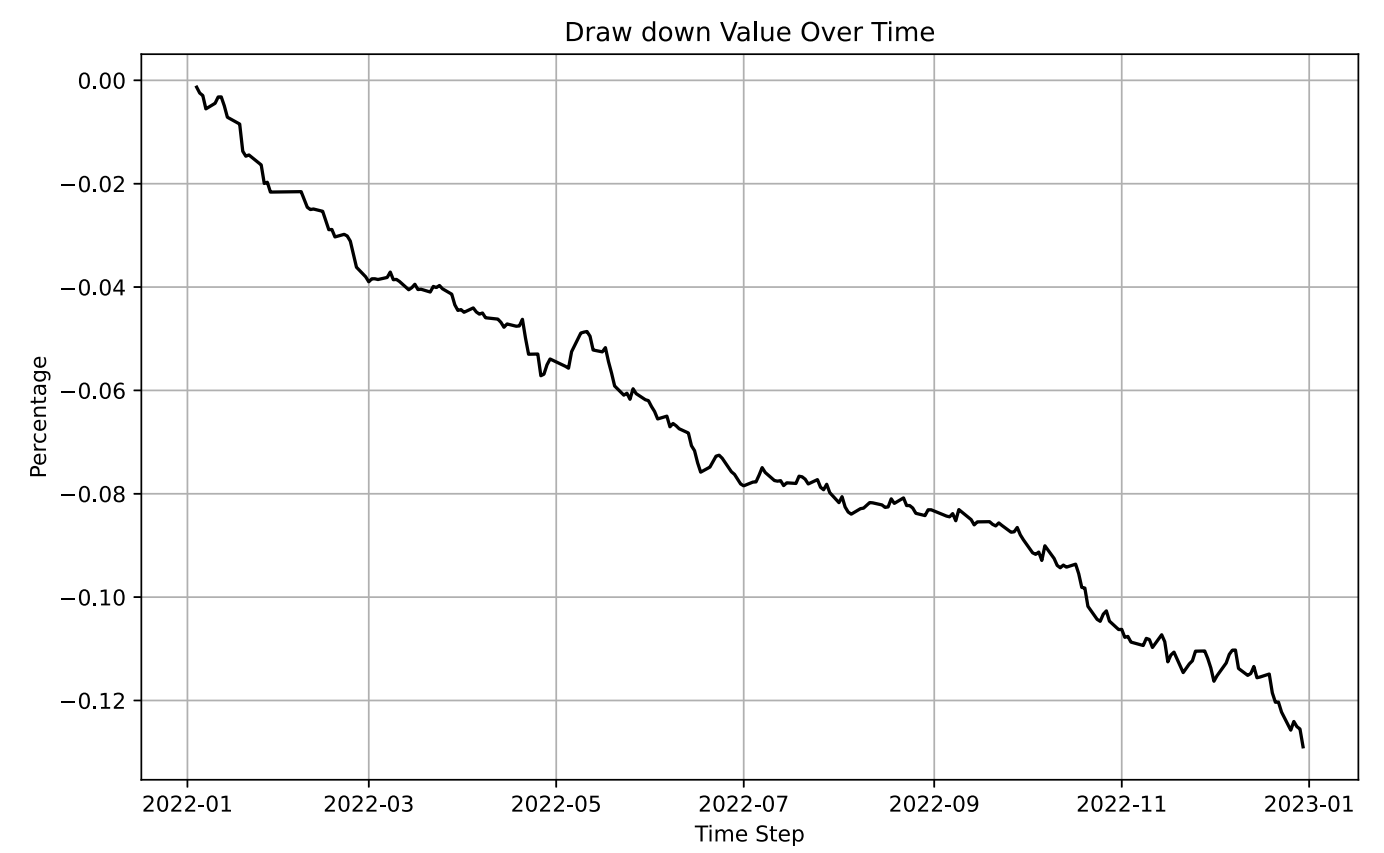
In-sample Backtesting Period

- **From:** 2022-01-01 — **To:** 2023-01-01

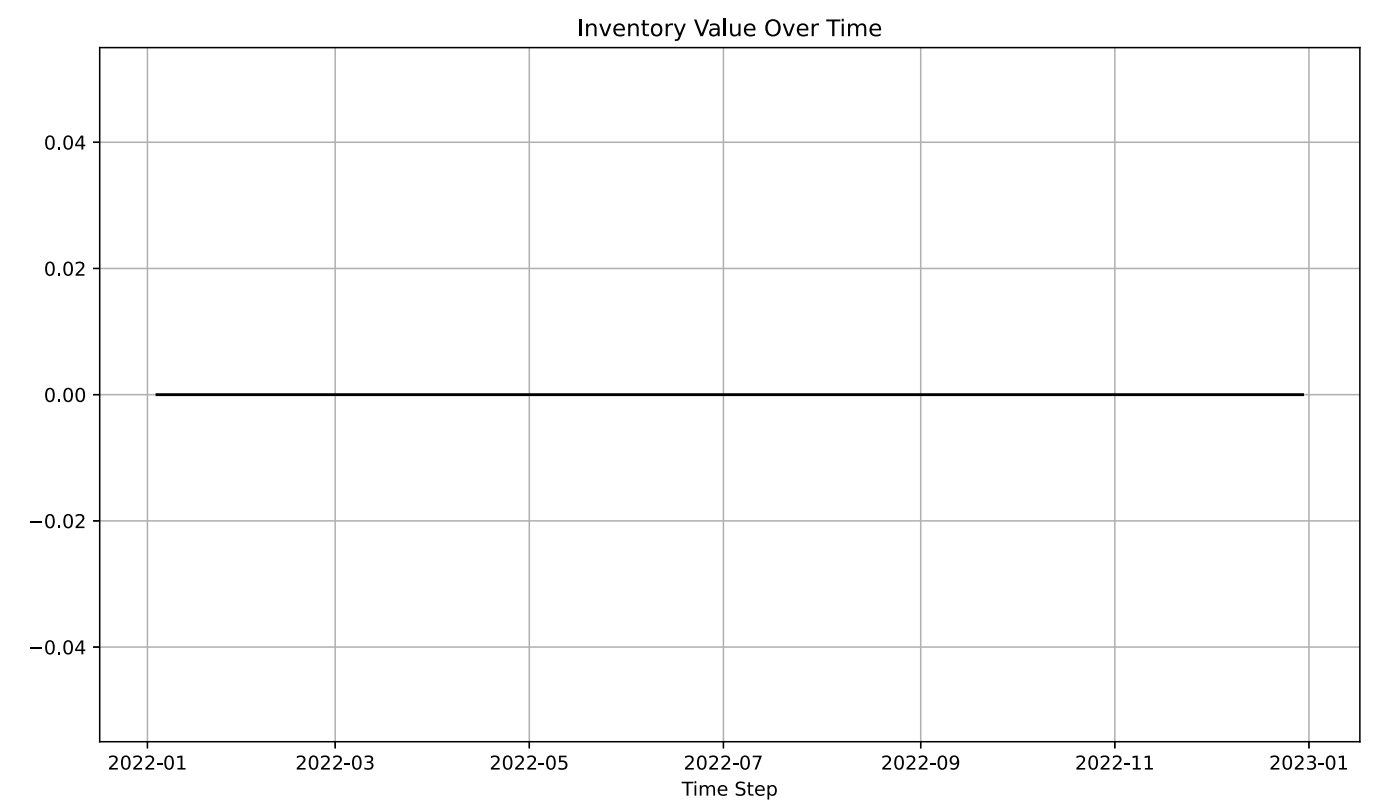
HPR Chart



Drawdown Chart



Inventory Chart



Optimization

Configuration is stored in `parameter/optimization_parameter.json`. The random seed ensures reproducibility. Run:

```
python optimization.py
```

The currently found optimized parameters with seed `2025`:

```
{
  "step": 3.1
}
```

Out-of-sample Backtesting

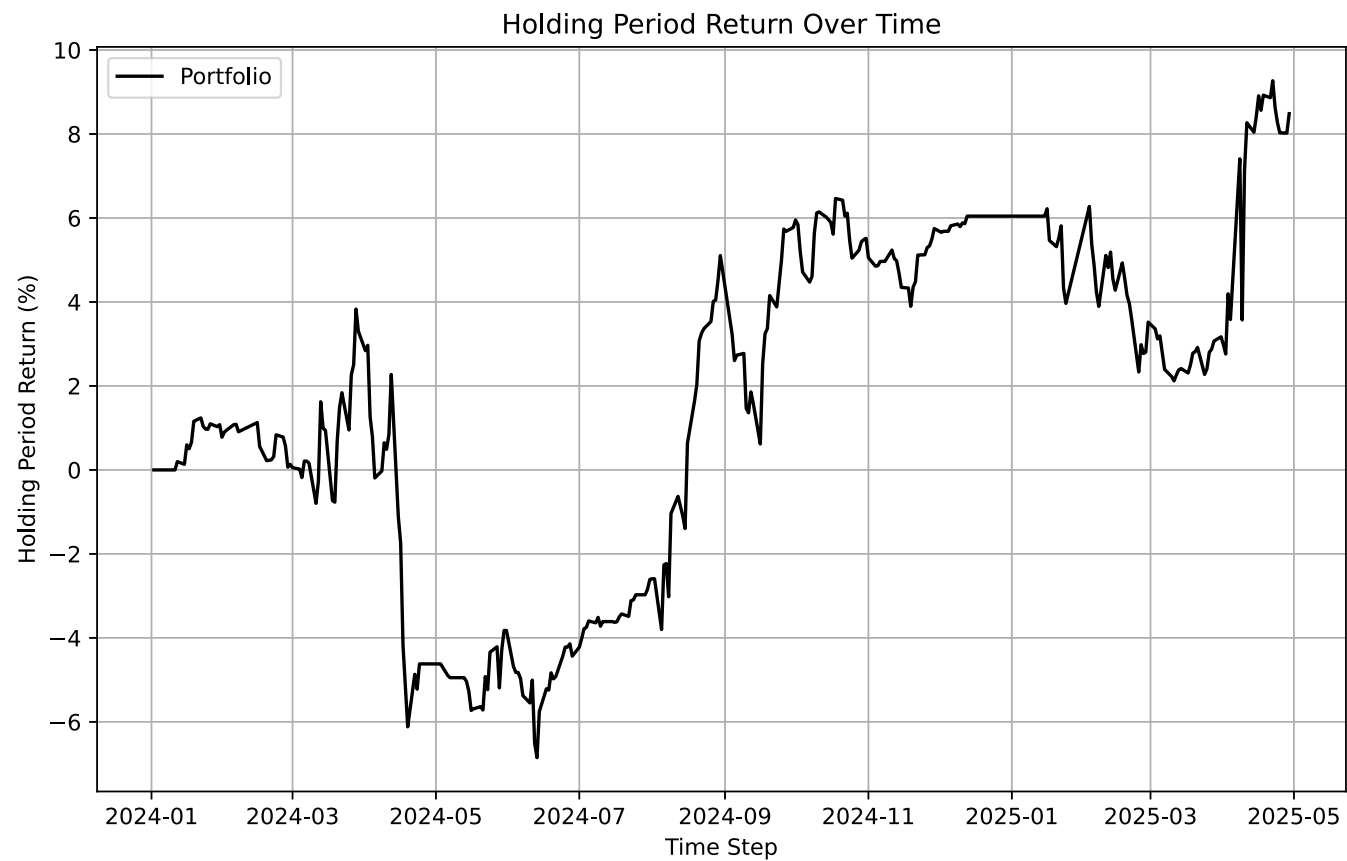
Uses the optimized parameters from `parameter/optimized_parameter.json`. Run:

```
python evaluation.py
```

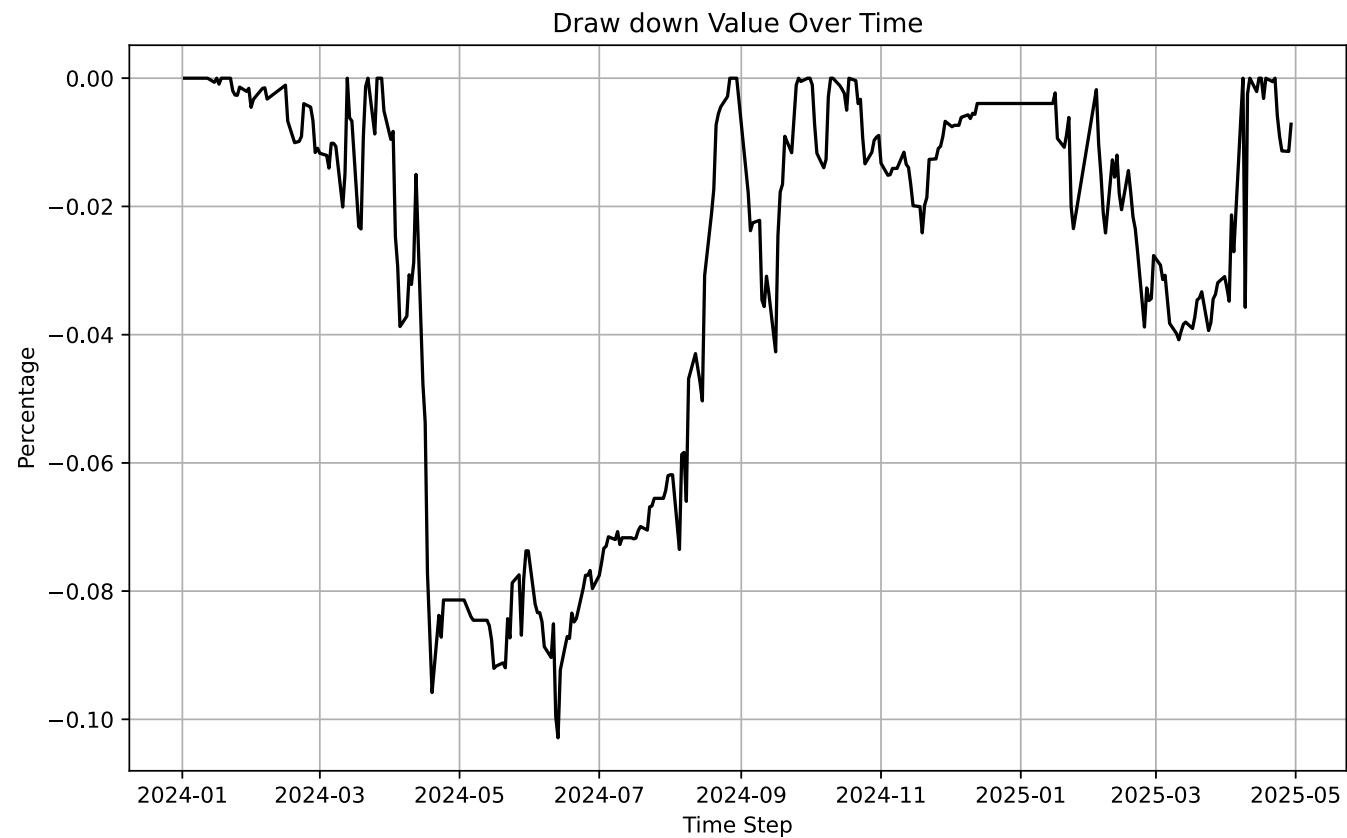
Out-of-sample Backtesting Period

- **From:** 2024-01-02 — **To:** 2025-04-29

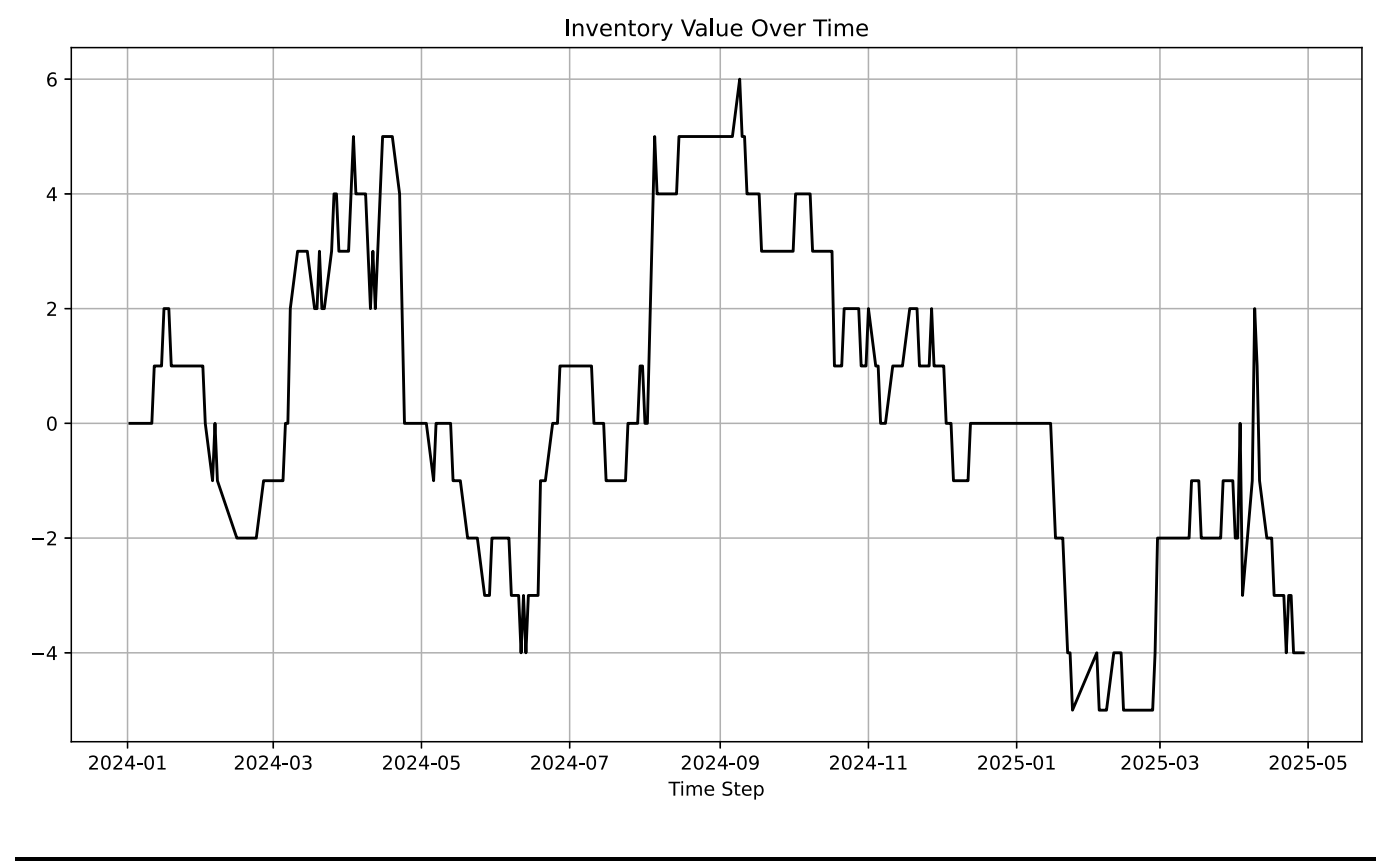
HPR Chart



Drawdown Chart



Inventory Chart



Logging

The bot logs every tick evaluation with full state context:

```
[10:00:01] [INFO] TICK EVAL | Price: 1285.0 | SMA: 1284.50 | Window: 10/10
| Pos: 0 | Lock: None
[10:00:02] [INFO] 📈 BUY Signal (Bullish Crossover)
[10:00:02] [INFO] 📄 Sent BUY order: a3f1bc92 at 1286.0
[10:00:02] [INFO] ✅ ORDER FILLED: 1 @ 1285.5
```

Reference

[1] ALGOTRADE, *Algorithmic Trading Theory and Practice — A Practical Guide with Applications on the Vietnamese Stock Market*, 1st ed. DIMI BOOK, 2023. Accessed: Apr. 30, 2025. [Online]. Available: [Link](#)

[2] ALGOTRADE, "AlgoTrade PaperBroker Platform," [Website](#). Available: [Link](#) (accessed Apr. 30, 2025).